Prelude iOS — Product Requirements Document

On-Device AI Therapy Prep Agent — Apple Intelligence Architecture *Living document. Updated by coding agent as build progresses.*

Document Metadata

Field	Value
Product Name	Prelude iOS
Version	1.0
Platform	iOS 26+ / iPhone with Apple Intelligence
Minimum Device	iPhone 15 Pro (A17 Pro)
AI Runtime	Foundation Models (on-device, zero API cost)
Last Updated	—
Build Status	🔴 Not Started

Table of Contents

1. Product Overview
2. Core Principles
3. Target Devices & Availability
4. Core Features
5. Foundation Models Architecture
6. Agent System
7. Voice System
8. Memory Schema

9. Safety Rules
10. Design System
11. Swift File Architecture
12. Build Phases & Task Tracker
13. App Store & Privacy Requirements
14. Known Risks & Mitigations

---

1. Product Overview

Prelude is a private, on-device, voice-first reflection agent that helps users prepare for therapy sessions. It guides a natural conversation, extracts emotional themes, and generates a structured personal brief the user can carry into their session.

What it is:

- A reflection engine
- A preparation tool
- An insight extractor
- A conversation guide
- A longitudinal memory system

What it is not: therapy, diagnosis, or medical advice.

Why on-device: Zero API cost means unlimited free use. All conversation data stays on the device — never leaves, never touches a server. This is a stronger privacy guarantee than any cloud product can make, and it is the core trust proposition of Prelude iOS.

Relationship to Prelude Web: Prelude iOS is a spiritual successor to the web version. Same conversation philosophy, same brief format, same emotional design intent — rebuilt natively for iPhone with on-device AI. The two products can coexist and share users who want the iOS experience.

---

2. Core Principles

- **On-device AI only** — Foundation Models, no external API calls

- **Voice first** — the primary interaction is speaking, not typing
- **Agentic, not chatbot** — the agent drives the conversation with purpose, not just responds
- **Tool-gated memory** — the model never writes to storage directly, only through typed tools
- **Availability-aware** — every code path handles the model being unavailable gracefully
- **Privacy absolute** — no analytics, no telemetry, no network calls during a session
- **Design with intention** — every visual and haptic choice serves the emotional register of the product

---

3. Target Devices & Availability

Supported Devices

Apple Intelligence requires Apple Silicon with at least 8GB unified memory.

Device	Supported
iPhone 15 Pro / Pro Max	✅
iPhone 16 / 16 Plus / 16 Pro / Pro Max	✅
iPhone 17 series	✅
iPhone 15 (non-Pro)	❌ (A16, no Apple Intelligence)
iPad with M-series chip	✅ (future consideration)

Availability States

Foundation Models can be unavailable even on supported hardware. Every inference call must check availability and handle each state gracefully — **no crashes, no silent failures.**

```
enum ModelAvailabilityState {
    case available
    case notSupported          // device does not support Apple Intelligence
    case disabled              // Apple Intelligence turned off in Settings
```

```
    case downloading           // model downloading in background
    case lowPower              // Low Power Mode active
    case thermalThrottle       // device too hot
    case unknown
}
```

Required guard pattern — use before every session start:

```
let availability = SystemLanguageModel.default.availability

switch availability {
case .available:
    startSession()
case .unavailable(.appleIntelligenceNotEnabled):
    showOnboarding(.enableAppleIntelligence)
case .unavailable(.modelNotReady):
    showOnboarding(.modelDownloading)
default:
    showTemporaryUnavailableState()
}
```

User-facing states (not error messages — warm, clear, helpful):

- Not supported: "Prelude requires Apple Intelligence. It's available on iPhone 15 Pro and later."
- Disabled: "Turn on Apple Intelligence in Settings → Apple Intelligence & Siri to use Prelude."
- Downloading: "Prelude is getting ready — Apple Intelligence is setting up in the background. This only happens once."
- Low Power: "Connect to power to start a session — Prelude needs full performance to run."
- Thermal: "Your iPhone needs a moment to cool down. Prelude will be ready shortly."

---

4. Core Features

F1 — Live Voice Session

The primary experience. User speaks naturally. Agent listens, reflects, asks, and extracts.

Session flow:

```
User taps "Begin" →
Mic activates →
SpeechAnalyzer streams transcript →
Silence detected (800ms threshold) →
Agent processes with LanguageModelSession →
Tool calls execute (saveInsight, tagEmotion, etc.) →
Agent response text generated →
Premium TTS speaks response →
Mic reactivates →
Loop continues until agent or user ends session
```

Session duration: 8–12 minutes target. Agent guides pacing. No hard cutoff.

F2 — Agentic Conversation Engine

Tool-based agent loop. Not a chatbot. The agent has a goal: surface what the user needs to bring to therapy. It pursues that goal through structured questions and reflection, not open-ended chat.

See Section 6 for full agent architecture.

F3 — Insight Extraction

During session, the agent calls `saveInsight()` silently when it detects emotionally significant content. These are never shown to the user mid-session — they accumulate invisibly and become the raw material for the session brief.

Insight dimensions:

- Theme (what it's about)
- Emotion (how it feels)
- Concern (what's unresolved)
- Goal (what outcome the user wants)
- Conflict (tension between desires or people)

F4 — Session Brief

Generated after every session. Not a transcript summary — a personal brief written in the user's voice. Five to seven structured cards covering:

1. How I showed up today (emotional state)

2. The thing that's really weighing on me
3. Key emotion underneath it
4. What I want to make sure I say
5. An unresolved thread worth exploring
6. What I'm hoping therapy helps with today
7. Pattern note (if recurring theme detected across 3+ sessions)

F5 — Session History & Emotional Patterns

- Chronological list of past sessions with brief previews
- Emotional arc visualization per session (how tone shifted)
- Pattern detection across sessions (themes recurring over weeks)
- Weekly brief combining multiple sessions into a single view

F6 — Weekly Brief

Generated automatically after the week's sessions. Surfaces:

- Recurring themes across the week
- Emotional patterns (what dominated, what shifted)
- One reflection prompt for the upcoming session

F7 — Memory System

Local only. SwiftData. Structured storage for sessions, insights, cards, and briefs. No iCloud sync in V1. Data stays on device. See Section 8.

---

5. Foundation Models Architecture

> **Critical reading for the coding agent.** This section corrects the common misunderstanding of Foundation Models as a generic LLM API. Apple's framework has specific patterns that must be followed.

The Actual Framework: LanguageModelSession

The entry point for all inference is `LanguageModelSession`, not a custom wrapper:

```
import FoundationModels

let session = LanguageModelSession(
    model: .default,
    tools: [
        SaveInsightTool(),
        TagEmotionTool(),
        GenerateCardTool(),
        SummarizeSessionTool(),
        GetPastInsightsTool(),
        EndSessionTool()
    ],
    instructions: Instructions.sessionAgent  // from PromptBuilder
)
```

The model is Apple's on-device 3B model. It is accessed via `SystemLanguageModel.default`. Do not attempt to specify model names or sizes — there is only one model available via this API.

Structured Output: @Generable

Do not prompt the model to return JSON and then parse it. Use `@Generable` to constrain output to typed Swift structs. This is more reliable, type-safe, and idiomatic:

```
import FoundationModels

@Generable
struct AgentDecision {
    @Guide("The action the agent should take next")
    var action: AgentAction

    @Guide("The text the agent should speak to the user. Warm, calm, and brief.")
    var spokenResponse: String

    @Guide("Internal reasoning — not spoken aloud")
    var reasoning: String
}

@Generable
enum AgentAction: String {
    case respond           // speak and continue listening
    case askQuestion       // ask a targeted follow-up
    case saveInsight       // save an insight (triggers tool call)
    case reflectBack       // mirror the emotional weight back
    case readBackSummary   // read the brief back to the user
    case endSession        // gracefully close
```

```
}
```

Tool Calling: Tool Protocol

The model calls tools natively through the session. Each tool conforms to Apple's `Tool` protocol:

```
import FoundationModels

struct SaveInsightTool: Tool {
    let name = "saveInsight"
    let description = "Save an emotionally significant insight from the conversat

    @Generable
    struct Arguments {
        @Guide("The insight text, written as a first-person statement from the us
        var text: String

        @Guide("The primary emotion detected")
        var emotion: EmotionLabel

        @Guide("The thematic category")
        var theme: String

        @Guide("Importance level 1-3")
        var importance: Int
    }

    func call(arguments: Arguments) async throws -> ToolOutput {
        let insight = Insight(
            text: arguments.text,
            emotion: arguments.emotion,
            theme: arguments.theme,
            importance: arguments.importance,
            sessionId: SessionStore.shared.currentSessionId
        )
        await MemoryStore.shared.saveInsight(insight)
        return ToolOutput("Insight saved.")
    }
}
```

Prompt Design for a 3B Model

The Foundation Models on-device model is ~3 billion parameters. This requires different prompt engineering than cloud models. Key rules:

Be directive, not conversational in system prompts. The model performs better with explicit role definitions and format constraints than with flowing prose instructions.

Keep the context window lean. Unlike cloud models, context is expensive on-device. Do not include the full raw transcript in every call — pass a rolling summary of the last 3–4 exchanges plus the current turn.

Constrain output length. The model can over-generate. Use `@Guide` annotations to specify expected lengths.

Test every prompt on device. Prompts that work in Playground may behave differently on the actual 3B model. Validate on hardware.

---

6. Agent System

LanguageModelSession Lifecycle

One session per conversation. Do not create a new session per turn — the session maintains conversation state internally.

```
class AgentController: ObservableObject {
    private var modelSession: LanguageModelSession?
    private var conversationPhase: ConversationPhase = .warmOpen

    func startSession() async throws {
        guard SystemLanguageModel.default.availability == .available else {
            throw AgentError.modelUnavailable
        }

        modelSession = LanguageModelSession(
            model: .default,
            tools: ToolRegistry.allTools,
            instructions: PromptBuilder.systemPrompt(for: conversationPhase)
        )
    }

    func process(transcript: String) async throws -> AgentDecision {
        guard let session = modelSession else {
            throw AgentError.sessionNotStarted
        }

        let prompt = PromptBuilder.buildTurnPrompt(
            transcript: transcript,
            phase: conversationPhase,
```

```
            recentInsights: await MemoryStore.shared.getRecentInsights(limit: 5)
        )

        let response = try await session.respond(
            to: prompt,
            generating: AgentDecision.self
        )

        await updatePhase(based: response.content.action)
        return response.content
    }
}
```

Conversation Phases

```
enum ConversationPhase {
    case warmOpen       // ~60s — greeting, set intention
    case openField      // ~3-4min — open question, listen broadly
    case excavation     // ~3-4min — follow the emotional thread
    case readBack       // ~2min — summarize aloud, confirm
    case closing        // ~30s — warm close
}
```

Phase transitions are managed by `AgentController` based on elapsed time and agent action signals. The agent does not know its phase number — it receives phase-appropriate instructions in the system prompt.

Tool Registry

```
enum ToolRegistry {
    static var allTools: [any Tool] {
        [
            SaveInsightTool(),
            TagEmotionTool(),
            GenerateCardTool(),
            GetPastInsightsTool(),
            CheckPatternsTool(),
            SummarizeSessionTool(),
            EndSessionTool()
        ]
    }
}
```

Model never writes to SwiftData directly. Only tools write to storage. This keeps the

data layer clean and testable independently of the model.

System Prompt Architecture

Prompts are phase-sensitive and short. Assembled by `PromptBuilder`:

```
struct PromptBuilder {
    static func systemPrompt(for phase: ConversationPhase) -> String {
        let base = """
        You are Prelude, a warm and attentive reflection guide.
        Your goal: help the user identify what they most need to bring to therapy
        You are NOT a therapist. You do NOT diagnose or advise.
        You ask, reflect, and listen.
        Keep responses under 3 sentences. Speak like a thoughtful human, not an A
        Return your response as a structured AgentDecision.
        """

        let phaseInstruction = switch phase {
        case .warmOpen:
            "Phase: OPEN. Greet warmly. Set intention. Ask one open question abou
        case .openField:
            "Phase: LISTEN. Ask one broad question. After response, reflect the e
        case .excavation:
            "Phase: EXCAVATE. Follow the highest-weight emotional thread. Ask: wh
        case .readBack:
            "Phase: SUMMARIZE. Read back three themes and their emotional state.
        case .closing:
            "Phase: CLOSE. Warmly end the session. Generate brief."
        }

        return base + "\n\n" + phaseInstruction
    }
}
```

Agent Loop

```
// In AgentController
func runAgentLoop(userSpeech: String) async {
    do {
        // 1. Get agent decision
        let decision = try await process(transcript: userSpeech)

        // 2. Speak the response
        await VoiceEngine.shared.speak(decision.spokenResponse)
```

```
        // 3. Handle action
        switch decision.action {
        case .endSession:
            await generateBrief()
            await endSession()
        case .readBackSummary:
            await transitionToPhase(.readBack)
        default:
            // Continue listening
            await VoiceEngine.shared.activateMic()
        }

    } catch AgentError.modelUnavailable {
        await handleModelUnavailable()
    } catch {
        await handleGenericError(error)
    }
}
```

---

7. Voice System

Frameworks

Framework	Purpose
`SpeechAnalyzer`	Streaming speech-to-text (iOS 18+, replaces SFSpeechRecognizer)
`AVAudioEngine`	Audio capture and processing pipeline
`AVSpeechSynthesizer`	Text-to-speech output
`CoreHaptics`	Haptic feedback for session states

SpeechAnalyzer Integration

`SpeechAnalyzer` provides streaming partial results and confirmed final results. **Only fire the agent on final results.** Partials are for UI display only.

```
import SpeechAnalysis

class SpeechRecognizer {
```

```
    private var analyzer: SpeechAnalyzer?
    private var silenceTimer: Timer?
    private let silenceThreshold: TimeInterval = 0.8

    func startListening() async throws {
        let session = SpeechAnalyzer.Session()

        for try await result in session.results {
            switch result {
            case .partial(let transcript):
                // Update UI transcript display only
                await MainActor.run {
                    TranscriptViewModel.shared.updatePartial(transcript.text)
                }

            case .final(let transcript):
                // Cancel silence timer — we have a final result
                silenceTimer?.invalidate()
                // Fire agent
                await AgentController.shared.runAgentLoop(userSpeech: transcript.
            }
        }
    }

    // Silence detection: if partial results stop updating for 800ms, treat as fi
    private func resetSilenceTimer(text: String) {
        silenceTimer?.invalidate()
        silenceTimer = Timer.scheduledTimer(withTimeInterval: silenceThreshold, r
            Task {
                await AgentController.shared.runAgentLoop(userSpeech: text)
            }
        }
    }
}
```

TTS — Premium Voice Requirement

Do not use default `AVSpeechSynthesizer` voices. They sound robotic and will undermine the entire emotional register of the product.

Use Premium system voices — on-device, Siri-quality, free:

```
class TTS {
    private let synthesizer = AVSpeechSynthesizer()

    // Preferred voices in priority order
```

```
    private let preferredVoiceIdentifiers = [
        "com.apple.voice.premium.en-US.Zoe",     // warm, female
        "com.apple.voice.premium.en-US.Evan",    // warm, male
        "com.apple.voice.enhanced.en-US.Zoe",    // fallback enhanced
        "com.apple.voice.enhanced.en-US.Evan"    // fallback enhanced
    ]

    func speak(_ text: String) async {
        let voice = preferredVoiceIdentifiers
            .compactMap { AVSpeechSynthesisVoice(identifier: $0) }
            .first ?? AVSpeechSynthesisVoice(language: "en-US")

        let utterance = AVSpeechUtterance(string: text)
        utterance.voice = voice
        utterance.rate = 0.48          // slightly slower than default — calm, un
        utterance.pitchMultiplier = 0.95
        utterance.volume = 0.9

        // Mute mic while agent speaks to prevent loopback
        await SpeechRecognizer.shared.pauseListening()
        synthesizer.speak(utterance)

        // Resume mic after speech ends + 350ms buffer
        await waitForSpeechEnd()
        try? await Task.sleep(nanoseconds: 350_000_000)
        await SpeechRecognizer.shared.resumeListening()
    }
}
```

Turn-Taking State Machine

```
enum VoiceState {
    case idle
    case listening          // mic active, SpeechAnalyzer streaming
    case processing         // agent running, mic muted
    case speaking           // TTS active, mic muted
    case interrupted        // user spoke during agent speech
    case paused             // user tapped pause
    case ended              // session complete
}
```

Transitions:

- `idle → listening` **: user taps Begin**
- `listening → processing` **: 800ms silence detected after final transcript**

- `processing → speaking` **: agent decision received**
- `speaking → listening` **: TTS ends + 350ms**
- `any → paused` **: user taps pause button**
- `any → ended` **: agent calls** `EndSessionTool` **or user taps end**

---

8. Memory Schema

Local only. SwiftData. No iCloud sync in V1.

```
import SwiftData

@Model
class Session {
    var id: UUID
    var startedAt: Date
    var completedAt: Date?
    var durationSeconds: Int
    var phase: String                    // final phase reached
    var insights: [Insight]
    var cards: [SessionCard]
    var brief: SessionBrief?
    var emotionalArc: EmotionalArc?
}

@Model
class Insight {
    var id: UUID
    var text: String
    var emotion: String                  // EmotionLabel raw value
    var theme: String
    var importance: Int                  // 1-3
    var sessionId: UUID
    var timestamp: Date
}

@Model
class SessionCard {
    var id: UUID
    var type: CardType                   // .mainConcern, .keyEmotion, .therapyQu
    var text: String
    var sessionId: UUID
```

```
}

@Model
class SessionBrief {
    var id: UUID
    var sessionId: UUID
    var generatedAt: Date
    var emotionalState: String
    var themes: [String]
    var patientWords: String             // near-verbatim quote preserved
    var focusItems: [String]
    var patternNote: String?
}

@Model
class WeeklyBrief {
    var id: UUID
    var weekStart: Date
    var summary: String
    var themes: [String]
    var dominantEmotion: String
    var suggestions: [String]
    var sessionIds: [UUID]
}

@Model
class EmotionalArc {
    var openingEmotion: String
    var peakIntensityTimestamp: Date?
    var closingEmotion: String
    var dominantEmotion: String
    var emotionSequence: [String]        // ordered list of tagged emotions throu
}

enum CardType: String, Codable {
    case emotionalState
    case mainConcern
    case keyEmotion
    case whatToSay
    case unresolvedThread
    case therapyGoal
    case patternNote
}

enum EmotionLabel: String, Codable, CaseIterable {
    case anxious, sad, angry, confused, hopeful, overwhelmed, frustrated, neutral
}
```

9. Safety Rules

What the Agent Must Never Do

- Diagnose any condition
- Suggest medications or treatments
- Interpret symptoms clinically
- Encourage the user to rely on Prelude instead of seeing a therapist
- Continue probing when a user expresses acute distress or hopelessness

What the Agent Must Always Do

- De-escalate and transition to closing when distress language is detected
- Surface crisis resources when distress is severe (see below)
- Frame itself as a preparation and reflection tool, not a therapeutic one

Crisis Detection & Response

In `PromptBuilder`, include a safety override instruction:

```
SAFETY OVERRIDE: If the user expresses thoughts of self-harm, suicide,
hopelessness, or acute crisis — immediately:
1. Acknowledge warmly without probing further
2. Set action to endSession
3. Set spokenResponse to the crisis acknowledgment template
```

Crisis acknowledgment template (spoken by TTS):

> "I hear that things feel really heavy right now. I'm not the right support for what you're describing — but support is available. Please reach out to the 988 Suicide and Crisis Lifeline by calling or texting 988. They're there for exactly this."

After this, the session ends automatically and the app shows the 988 resource card on screen.

In-App Disclaimer

Shown on first launch and accessible from Settings:

> "Prelude is a personal reflection and preparation tool. It is not therapy, and it is not a substitute for professional mental health care. If you are in crisis, please contact the 988 Suicide & Crisis Lifeline (call or text 988)."

---

10. Design System

> This section is prescriptive. The visual and haptic design of Prelude iOS is as important as its technical architecture. A therapy prep app lives or dies by how it makes the user *feel* in the first 10 seconds — before they've said a single word.

10.1 The Design Problem with Most AI Apps

Most AI apps look like they were designed by the AI. They share a visual vocabulary that has become instantly recognizable and equally instantly forgettable: purple-to-blue gradients, pulsing orbs for "thinking," waveform equalizers, dark backgrounds with glowing UI elements, robotic grid layouts, and a cold, clinical use of space that feels like a data dashboard rather than a conversation.

Prelude must feel like none of those things. It must feel like something the user would pick up and hold.

10.2 The Design Direction: Warm Instrument

The conceptual anchor for Prelude iOS is a **warm instrument** — something between a leather-bound journal and a finely tuned musical instrument. It is analog in spirit, precise in execution, and intimate by design.

References that inform this direction:

- **Day One** — the warmth of digital journaling done right
- **Things 3** — precision and calm in a productivity app
- **Endel** — generative atmosphere that serves a mood
- **A beautiful analog object** — a Moleskine, a cello, a well-worn compass

References to explicitly NOT reference:

- Any chatbot UI (OpenAI, Gemini, Claude)
- The Calm or Headspace app (spa-generic)
- VisionOS spatial computing aesthetics (wrong register)

- Any app that uses “frosted glass” as a personality

10.3 Color System

Foundation Palette

The palette is built on warm earth tones with a single cool accent. It is not a mental health pastel palette. It is richer, more grounded — the colors of parchment, aged wood, amber light, and deep forest shadow.

```
// Semantic color tokens — define in Assets.xcassets with dark/light variants

// Backgrounds
Color.preludeDepth          // Dark: #0F0D0A  Light: #FAF7F2  — near-black warm /
Color.preludeSurface        // Dark: #1C1813  Light: #F0EBE3  — card/panel backgr
Color.preludeRaised         // Dark: #252018  Light: #E8E1D6  — elevated surfaces

// Text
Color.preludePrimary        // Dark: #F5F0E8  Light: #1A1612  — main text
Color.preludeSecondary      // Dark: #9E9485  Light: #6B6057  — secondary text
Color.preludeTertiary       // Dark: #5C5448  Light: #9E9485  — hints, labels

// Accent — used sparingly, maximum two uses per screen
Color.preludeAmber          // #C8873A — warm amber, primary accent
Color.preludeSage           // #7A9E7E — muted sage green, secondary accent

// States — emotional state colors, subtle tints not full fills
Color.preludeCalm           // #4A7C8E — deep teal, used for listening state
Color.preludeActive         // #C8873A — amber, used for speaking state
Color.preludeProcessing     // #6B5E4E — warm brown, used for thinking state
```

Color Rules

Never use pure black or pure white — the palette uses warm-shifted near-blacks and near-whites only.

Accent colors (`preludeAmber` , `preludeSage`) appear on a maximum of two elements per screen. If everything glows amber, nothing does.

The background color shifts subtly based on session state — a barely perceptible warm tint when the user is speaking, a cool-neutral tint when the agent is speaking. This is ambient emotional feedback, not a UI state indicator. The shift happens over 1.5 seconds via a `withAnimation(.easeInOut(duration: 1.5))` on the background color.

10.4 Typography

Type System

```
// Three typefaces only. Each has a specific role.

// 1. New York — for emotionally significant content
//    Used for: session brief content, insight cards, the agent's spoken words di
//    Weight: Regular and Semibold only
//    Never used for: UI labels, buttons, navigation

// 2. SF Pro — for UI and informational content
//    Used for: navigation, settings, timestamps, metadata
//    Weight: Regular and Medium only

// 3. SF Mono — for the live transcript only
//    Used for: the scrolling transcript during a session
//    Creates separation between "what was said" and "what the UI is doing"
//    Weight: Regular only, reduced opacity (0.7)
```

Type Scale

```
// Dynamic Type compatible — all sizes respond to user's accessibility settings

extension Font {
    static let preludeHero = Font.custom("NewYork", size: 34).weight(.semibold)
    static let preludeTitle = Font.custom("NewYork", size: 24).weight(.regular)
    static let preludeCardTitle = Font.custom("NewYork", size: 19).weight(.semibo
    static let preludeCardBody = Font.custom("NewYork", size: 16).weight(.regular
    static let preludeLabel = Font.system(size: 13, weight: .medium, design: .def
    static let preludeCaption = Font.system(size: 11, weight: .regular, design: .
    static let preludeTranscript = Font.system(size: 14, weight: .regular, design
}
```

Typography Rules

Emotional weight is communicated through type size, not color. When the agent surfaces something significant — a pattern note, the user's own words preserved — it appears larger and in New York, not in a colored badge.

Line spacing is generous: 1.6× line height minimum for New York body text. The text should breathe.

Never use all-caps except for timestamp labels. It reads as shouting.

10.5 The Session Screen — The Heart of the App

This is the screen that most AI apps get wrong. Here is exactly how Prelude's session screen works.

What It Is NOT

- Not a chat bubble interface
- Not a waveform equalizer (bars going up and down like a music app)
- Not a pulsing orb
- Not a full-screen transcript

What It IS

The session screen has two zones:

Zone 1 — The Presence (top 60% of screen)

A large, breathing organic shape in the center of the screen. Not a circle. Not an orb. A soft, irregular form — like an ink drop suspended in water, or the abstract shape of a breath.

This shape is built with `Canvas` or `TimelineView` + SwiftUI paths. Its behavior:

- **Idle/Listening:** Slow, barely perceptible expansion and contraction (1 breath per ~4 seconds). Color: `preludeCalm`. Opacity: 0.15.
- **User speaking:** Shape responds to vocal amplitude via `AVAudioEngine` tap. It breathes faster and more expansively, but organically — not in sync with individual phonemes like an equalizer. Color tints toward `preludeActive`. Think: the shape *hearing* you, not *displaying* your voice.
- **Processing:** Shape contracts gently and holds still. Color tints to `preludeProcessing`. No spinner. No "thinking..." text. The stillness IS the communication.
- **Agent speaking:** Shape expands slowly and holds a fuller form. Color: `preludeCalm`. A gentle, soft pulse in time with TTS speech rhythm.

The shape must never feel mechanical. Implemented with `spring()` animations driven by amplitude readings averaged over 200ms windows — not raw real-time data. Raw

real-time data creates twitchy, robotic motion. Averaged data creates organic motion.

Zone 2 — The Ground (bottom 40% of screen)

Clean, dark surface. Two elements only:

1. **The current agent text** — what the agent just said, displayed in New York Regular, `preludePrimary` color, centered, 2-3 lines maximum. It fades in word by word in sync with TTS playback. It does not appear instantly — it arrives as it is spoken.
2. **The transcript scroll** — a small, low-opacity scrolling view showing the conversation in SF Mono. Barely visible. There for users who want it. Does not dominate the screen.

Three buttons at the very bottom edge, always visible:

- Pause (pause icon, minimal)
- End session (small text label, not an icon)
- Crisis resource (small "?" in bottom corner — not obvious, but always there)

No header. No navigation bar during a session. The session takes the full screen. The status bar hides.

10.6 Liquid Glass Usage Rules

Apple's Liquid Glass material should be used surgically, not as wallpaper.

Use Liquid Glass for:

- The session cards overlay when the brief is revealed
- Modal sheets (settings, disclaimer)
- The history list panel
- Navigation bar background when scrolling over content

Do not use Liquid Glass for:

- The session screen presence zone (it needs to feel alive, not glassy)
- Body text containers
- The main background (background has its own depth via color, not material)
- Any element that needs strong contrast for readability

Implementation rule: Every Liquid Glass surface must pass WCAG AA contrast (4.5:1)

for overlaid text. Test in both light and dark mode. Test on the actual device, not Simulator — Simulator renders materials differently.

```
// Correct Liquid Glass panel
RoundedRectangle(cornerRadius: 20)
    .fill(.ultraThinMaterial)
    .overlay(
        // Always add a subtle tint to ensure text contrast
        RoundedRectangle(cornerRadius: 20)
            .fill(Color.preludeSurface.opacity(0.4))
    )
    .shadow(color: .black.opacity(0.12), radius: 20, y: 8)
```

10.7 Motion Design

Principles

Motion in Prelude has a single rule: **motion communicates state, not decoration.** If a motion doesn't tell the user something, it doesn't exist.

Never use:

- Particle effects
- Confetti or celebration animations
- Looping background animations that serve no purpose
- Animations that interrupt the user's attention during a session

Use:

- State transitions (listening → processing → speaking)
- Content arrival (cards fading in as they're generated)
- Spatial navigation (sessions sliding in, not appearing)
- Ambient presence animation (the breathing shape)

Animation Tokens

```
extension Animation {
    static let preludeSpring = Animation.spring(response: 0.5, dampingFraction: 0
    static let preludeGentle = Animation.easeInOut(duration: 0.4)
    static let preludeAmbient = Animation.easeInOut(duration: 3.8).repeatForever(
```

```
    static let preludeReveal = Animation.easeOut(duration: 0.6)
}
```

Card Reveal

When the brief is generated after a session, cards do not all appear at once. They arrive sequentially, each with a 200ms delay:

```
ForEach(Array(cards.enumerated()), id: \.element.id) { index, card in
    BriefCard(card: card)
        .opacity(isVisible ? 1 : 0)
        .offset(y: isVisible ? 0 : 20)
        .animation(
            .preludeReveal.delay(Double(index) * 0.2),
            value: isVisible
        )
}
```

10.8 Haptic Design

Haptics are part of the UI. Used precisely, they make the app feel alive. Used carelessly, they feel like notifications.

```
enum PreludeHaptic {
    case sessionBegin    // soft, single tap — "I'm listening"
    case agentSpeaking   // subtle, slow heartbeat rhythm while agent speaks — pr
    case insightSaved    // imperceptible micro-tap — silent confirmation
    case briefReady      // medium, warm double-tap — "here is your brief"
    case sessionEnd      // gentle, slow fade-out pattern — "you're done"
    case error           // short, soft single — never harsh
}

// Implementation
class HapticEngine {
    private let engine: CHHapticEngine

    func play(_ haptic: PreludeHaptic) {
        switch haptic {
        case .sessionBegin:
            playPattern(intensity: 0.5, sharpness: 0.3, duration: 0.08)
        case .briefReady:
            playSequence([
                (intensity: 0.6, sharpness: 0.2, delay: 0.0, duration: 0.12),
                (intensity: 0.4, sharpness: 0.1, delay: 0.18, duration: 0.1)
            ])
```

```
            // etc.
            }
        }
    }
```

10.9 Screen-by-Screen Specifications

Home Screen

Layout:

- Large centered greeting in New York — "Good morning, [first name]." — at 55% from top
- Subtle timestamp below: "Thursday · Your next session is in 2 days" in SF Pro caption, `preludeSecondary`
- Single large tap target: "Begin Reflection" — not a button in the conventional sense. A text label in New York Semibold surrounded by generous invisible tap area. No border. No capsule. Amber color only.
- Below, minimal: last session summary in two lines, `preludeSecondary` color

What's NOT on the home screen: Navigation tabs, feature grids, progress metrics, streaks, badges, any gamification. The home screen does one thing — invites you to begin.

Background: Full-bleed gradient, very subtle — warm dark at edges, marginally lighter at center. The gradient shifts by time of day using `Date()`. Morning: warm honey tone. Evening: deep amber-brown. Not dramatic — barely perceptible. Implemented with `AngularGradient` with extremely low saturation.

Session Screen

Described in full in Section 10.5.

Additional detail: the session screen has no visible timer. Time pressure is the enemy of honest reflection. The user can see a subtle progress arc at the very edge of the presence shape — not a countdown, but a slow fill that completes at ~10 minutes. At ~8 minutes, the agent naturally begins transitioning toward the read-back phase regardless.

Brief Screen

The brief arrives as stacked cards on a dark surface. Cards are Liquid Glass panels with generous internal padding.

Each card has:

- A small icon in `preludeAmber` (SF Symbol, stroke style, not filled)
- A card type label in SF Pro caption, `preludeTertiary`
- The content in New York Regular, `preludePrimary`
- If the card contains the user's own words (the "What I want to make sure I say" card): those words appear in New York Italic, slightly larger, with a thin left border in `preludeAmber` — a visual signal that these are *their* words, preserved

Cards are swipeable. Swiping up reveals more cards. The stack feels like shuffling through physical notes.

At the bottom of the brief screen: "Take this to your session" — not a button, just a New York caption in `preludeSecondary`. Tapping it copies the brief as plain text for sharing.

History Screen

A vertical timeline. Each session is a single row:

- Date in SF Pro medium, left-aligned
- Dominant emotion as a small colored dot (emotion-to-color mapped)
- Session duration in SF Pro caption, right-aligned
- First theme from the brief, truncated, in New York Regular

Tapping reveals the full brief for that session, sliding in from the right.

No charts on the history screen. Emotional trend data is surfaced only in the Weekly Brief view. The history screen is for finding and reviewing specific sessions.

Weekly Brief Screen

One card, full-width, New York Semibold title: "This week." Below, three paragraphs of narrative prose — not bullet points — describing the week's emotional territory. A small section at the bottom: "Worth bringing up:" followed by one sentence in amber.

This screen generates once per week, after the week's first session. Tapping "Regenerate" updates it after subsequent sessions in the same week.

10.10 Accessibility

- All text uses Dynamic Type — never hardcoded font sizes
- Minimum tap target 44×44pt on all interactive elements
- VoiceOver labels on all custom shapes and views
- Reduce Motion: when enabled, the presence shape becomes a simple static ring that fades between states rather than animating
- High Contrast: Liquid Glass surfaces receive a stronger tint overlay to meet WCAG AA
- The crisis resource (988 link) is always VoiceOver accessible regardless of visual visibility

---

11. Swift File Architecture

```
Prelude/
├── App/
│   ├── PreludeApp.swift              — app entry, availability check
│   └── AppState.swift                — global state machine

├── Agent/
│   ├── AgentController.swift         — LanguageModelSession lifecycle, agent loo
│   ├── PromptBuilder.swift           — phase-sensitive system prompts
│   ├── AgentDecision.swift           — @Generable AgentDecision struct
│   └── ConversationPhase.swift       — phase enum and transition logic

├── Tools/
│   ├── ToolRegistry.swift            — allTools array
│   ├── SaveInsightTool.swift
│   ├── TagEmotionTool.swift
│   ├── GenerateCardTool.swift
│   ├── GetPastInsightsTool.swift
│   ├── CheckPatternsTool.swift
│   ├── SummarizeSessionTool.swift
│   └── EndSessionTool.swift

├── Voice/
│   ├── VoiceEngine.swift             — coordinates SpeechRecognizer + TTS
│   ├── SpeechRecognizer.swift        — SpeechAnalyzer wrapper, silence detection
│   ├── TTS.swift                     — AVSpeechSynthesizer, premium voice select
│   └── VoiceState.swift              — state machine enum
```

```
├── Memory/
│   ├── MemoryStore.swift            — SwiftData container and context
│   ├── SessionStore.swift           — session CRUD
│   ├── InsightStore.swift           — insight CRUD
│   ├── BriefStore.swift             — brief CRUD
│   └── PatternDetector.swift        — cross-session theme analysis

├── Models/
│   ├── Session.swift                — SwiftData @Model
│   ├── Insight.swift                — SwiftData @Model
│   ├── SessionCard.swift            — SwiftData @Model
│   ├── SessionBrief.swift           — SwiftData @Model
│   ├── WeeklyBrief.swift            — SwiftData @Model
│   ├── EmotionalArc.swift           — SwiftData @Model
│   ├── EmotionLabel.swift           — enum
│   └── CardType.swift               — enum

├── UI/
│   ├── Home/
│   │   ├── HomeView.swift
│   │   └── HomeViewModel.swift
│   ├── Session/
│   │   ├── SessionView.swift
│   │   ├── PresenceShape.swift      — the breathing organic shape
│   │   ├── TranscriptView.swift
│   │   └── SessionViewModel.swift
│   ├── Brief/
│   │   ├── BriefView.swift
│   │   ├── BriefCard.swift
│   │   └── BriefViewModel.swift
│   ├── History/
│   │   ├── HistoryView.swift
│   │   └── SessionRowView.swift
│   ├── Weekly/
│   │   ├── WeeklyBriefView.swift
│   │   └── WeeklyBriefViewModel.swift
│   └── Shared/
│       ├── PreludeColors.swift      — color tokens
│       ├── PreludeFonts.swift       — typography tokens
│       ├── PreludeAnimations.swift  — animation tokens
│       └── PreludeHaptics.swift     — haptic engine

├── Onboarding/
│   ├── OnboardingView.swift
│   ├── AvailabilityGateView.swift   — all ModelAvailabilityState cases
│   └── DisclaimerView.swift
```

```
└── Resources/
    ├── Assets.xcassets              — color assets, dark/light variants
    ├── Fonts/                       — New York is system-provided, no embed nee
    └── PrivacyInfo.xcprivacy        — privacy manifest (required by Apple)
```

---

12. Build Phases & Task Tracker

> **For the coding agent:** Update task status using:
>
> - ⬜ `TODO` — not started
> - 🔄 `IN PROGRESS` — currently being built
> - ✅ `DONE` — complete and tested on device
> - ❌ `BLOCKED` — blocked; note reason in Notes column
>
> **Critical:** Test every phase on a real device, not Simulator only. Foundation Models, SpeechAnalyzer, and haptics all behave differently on hardware.

Phase 1 — Voice Pipeline (Build This First)

Goal: Mic → transcript → TTS loop working on real hardware. No AI yet. **Rationale:** Voice is the highest-risk component. Prove it works before building anything that depends on it. **Status:** 🔴 Not Started

#	Task	Status	Notes
1.1	Create Xcode project, set deployment target iOS 26+, configure capabilities (Microphone, Speech Recognition)	⬜ TODO	
1.2	Set up `AVAudioEngine` audio capture pipeline	⬜ TODO	
1.3	Integrate `SpeechAnalyzer` for streaming transcription	⬜ TODO	
1.4	Implement partial vs final result handling — UI updates on partial, agent fires on final only	⬜ TODO	
	Implement silence detection: 800ms timer reset	⬜	

1.5	on each partial result	TODO	
1.6	Implement Premium TTS voice selection with fallback chain	⬜ TODO	Test all voices on device — Simulator voices differ
1.7	Implement mic mute during TTS playback + 350ms resume delay	⬜ TODO	Prevents loopback
1.8	Implement `VoiceState` state machine and transitions	⬜ TODO	
1.9	Build minimal test UI: mic button, transcript display, TTS output	⬜ TODO	Not the real UI — just for testing the pipeline
1.10	Test full voice loop on iPhone 15 Pro (real device, not Simulator)	⬜ TODO	Speak a sentence, hear it echoed via TTS. No AI yet.
1.11	Test mic/TTS loopback prevention — confirm no echo	⬜ TODO	

Phase 2 — Foundation Models Integration

Goal: LanguageModelSession working with @Generable output and availability handling. **Status:** 🔴 Not Started

#	Task	Status	Notes
2.1	Import FoundationModels framework, confirm availability on target device	⬜ TODO	
2.2	Implement `ModelAvailabilityState` enum and all availability checks	⬜ TODO	
2.3	Define `AgentDecision` @Generable struct and `AgentAction` enum	⬜ TODO	
2.4	Write `PromptBuilder` with phase-sensitive system prompts	⬜ TODO	Keep prompts short and directive — 3B model

2.5	Build `AgentController` with `LanguageModelSession` lifecycle	⬜ TODO	One session per conversation
2.6	Test basic text-in → AgentDecision-out loop	⬜ TODO	No voice, no tools yet — just model I/O
2.7	Validate all 4 conversation phases produce appropriate agent behavior	⬜ TODO	Test each phase prompt independently
2.8	Implement error handling for all model failure modes	⬜ TODO	

Phase 3 — Tool Registry

Goal: All tools implemented, model calling them correctly, memory writes gated through tools only. **Status:** 🔴 Not Started

#	Task	Status	Notes
3.1	Implement `SaveInsightTool`	⬜ TODO	
3.2	Implement `TagEmotionTool`	⬜ TODO	
3.3	Implement `GenerateCardTool`	⬜ TODO	
3.4	Implement `GetPastInsightsTool` — returns recent insights for context	⬜ TODO	
3.5	Implement `CheckPatternsTool` — detects recurring themes across sessions	⬜ TODO	
3.6	Implement `SummarizeSessionTool` — generates brief JSON	⬜ TODO	
3.7	Implement `EndSessionTool` — triggers brief generation and session close	⬜ TODO	
3.8	Register all tools in `ToolRegistry`	⬜ TODO	
		⬜	

3.9	Connect `LanguageModelSession` with full tool registry	TODO	
3.10	Test: agent correctly calls `saveInsight` during emotional content	⬜ TODO	
3.11	Test: agent correctly calls `endSession` after read-back phase	⬜ TODO	

Phase 4 — SwiftData Memory

Goal: All session data persists correctly. History loads accurately across app restarts.
Status: 🔴 Not Started

#	Task	Status	Notes
4.1	Define all SwiftData `@Model` classes (Session, Insight, SessionCard, SessionBrief, WeeklyBrief, EmotionalArc)	⬜ TODO	
4.2	Implement `MemoryStore` with ModelContainer and ModelContext	⬜ TODO	
4.3	Implement `SessionStore` CRUD operations	⬜ TODO	
4.4	Implement `InsightStore` CRUD operations	⬜ TODO	
4.5	Implement `BriefStore` CRUD operations	⬜ TODO	
4.6	Implement `PatternDetector` — cross-session theme frequency analysis	⬜ TODO	Flag themes appearing 3+ consecutive sessions
4.7	Wire tool calls to SwiftData writes	⬜ TODO	
4.8	Test data persistence across app restarts	⬜ TODO	
	Test pattern detection with simulated multi-	⬜	

4.9	session data	TODO	

Phase 5 — Full Agent Loop Integration

Goal: Complete voice + agent + tools + memory pipeline working end-to-end. **Status:** 🔴 Not Started

#	Task	Status	Notes
5.1	Connect `VoiceEngine` → `AgentController` (final transcript triggers agent)	⬜ TODO	
5.2	Connect `AgentController` → `TTS` (spokenResponse triggers speech)	⬜ TODO	
5.3	Connect `AgentController` → `ToolRegistry` → `MemoryStore`	⬜ TODO	
5.4	Implement phase transition timing logic	⬜ TODO	
5.5	Implement brief generation at session end	⬜ TODO	
5.6	Run a complete 10-minute session end-to-end	⬜ TODO	Real voice, real model, real memory writes
5.7	Validate brief quality across 3 different sessions	⬜ TODO	Adjust prompts as needed
5.8	Implement and test crisis detection + 988 resource response	⬜ TODO	

Phase 6 — Session Cards & Weekly Brief

Goal: Brief cards generated correctly. Weekly brief combines sessions accurately.
Status: 🔴 Not Started

#	Task	Status	Notes

6.1	Implement card generation logic — map brief fields to `CardType`	⬜ TODO	Max 7 cards per session
6.2	Implement `WeeklyBrief` generation logic	⬜ TODO	Triggered after each session if 2+ sessions in current week
6.3	Test cards with varied session content	⬜ TODO	
6.4	Test weekly brief with 3+ sessions in same week	⬜ TODO	

Phase 7 — Design Implementation

Goal: The app looks and feels exactly as specified in Section 10. Every screen. Every state. **Status:** 🔴 Not Started

#	Task	Status	Notes
7.1	Set up color token system in Assets.xcassets (dark + light variants for all tokens)	⬜ TODO	
7.2	Set up `PreludeFonts.swift` — New York, SF Pro, SF Mono scale	⬜ TODO	
7.3	Set up `PreludeAnimations.swift` — all animation tokens	⬜ TODO	
7.4	Implement `CHHapticEngine` wrapper in `PreludeHaptics.swift`	⬜ TODO	Test all haptic patterns on device
7.5	Build `PresenceShape.swift` — organic breathing shape with amplitude response	⬜ TODO	The most important UI component
7.6	Build `SessionView.swift` with two-zone layout	⬜ TODO	Full screen, no nav bar
7.7	Implement ambient background color shift per voice state	⬜ TODO	1.5s easeInOut transition
7.8	Implement agent text word-by-word reveal synced to TTS	⬜ TODO	

7.9	Build `HomeView.swift` — minimal, single-CTA layout	⬜ TODO	Time-of-day background gradient
7.10	Build `BriefView.swift` — stacked card reveal with sequential animation	⬜ TODO	
7.11	Build `BriefCard.swift` — Liquid Glass, New York content, amber icon	⬜ TODO	
7.12	Build `HistoryView.swift` — timeline layout	⬜ TODO	
7.13	Build `WeeklyBriefView.swift` — narrative prose layout	⬜ TODO	
7.14	Build `OnboardingView.swift` and all `AvailabilityGateView` states	⬜ TODO	
7.15	Implement Reduce Motion variants for all animations	⬜ TODO	
7.16	Implement High Contrast variants for all Liquid Glass surfaces	⬜ TODO	
7.17	Full design review on device — dark mode, light mode, Dynamic Type sizes	⬜ TODO	Do this on real hardware

---

Phase 8 — App Store Preparation

Goal: App store ready — privacy manifest, permission strings, App Store listing.
Status: 🔴 Not Started

#	Task	Status	Notes
8.1	Write `PrivacyInfo.xcprivacy` manifest — declare microphone, speech, local storage	⬜ TODO	Required by Apple since iOS 17
8.2	Write `NSMicrophoneUsageDescription` permission string	⬜ TODO	Warm and specific — explain why the mic is needed
8.3	Write `NSSpeechRecognitionUsageDescription`	⬜ TODO	

8.4	Confirm no network calls during session (Charles Proxy or Instruments)	⬜ TODO	Privacy promise must be technically verified
8.5	Build App Store screenshots for all required sizes	⬜ TODO	Show the session screen prominently
8.6	Write App Store description — wellness tool, not clinical, 988 in description	⬜ TODO	Category: Health & Fitness
8.7	Configure App Store privacy nutrition label — data not collected	⬜ TODO	Verify this is accurate
8.8	TestFlight beta build — test on 3+ real devices	⬜ TODO	
8.9	Submit for App Store review	⬜ TODO	Allow 1–7 day review window

---

13. App Store & Privacy Requirements

Privacy Manifest (PrivacyInfo.xcprivacy)

Required since iOS 17. Declare all data access:

```
<key>NSPrivacyAccessedAPITypes</key>
<array>
    <dict>
        <key>NSPrivacyAccessedAPIType</key>
        <string>NSPrivacyAccessedAPICategoryMicrophone</string>
        <key>NSPrivacyAccessedAPITypeReasons</key>
        <array>
            <string>1.0</string>
        </array>
    </dict>
</array>

<key>NSPrivacyCollectedDataTypes</key>
<array/>  <!-- No data collected — all stays on device -->

<key>NSPrivacyTracking</key>
<false/>
```

Permission Strings (Info.plist)

```
NSMicrophoneUsageDescription:
"Prelude listens to your voice during reflection sessions. Audio is processed
entirely on your device and is never sent anywhere."

NSSpeechRecognitionUsageDescription:
"Prelude converts your speech to text during sessions using on-device recognition
This never leaves your iPhone."
```

App Store Category & Rating

- **Primary category:** Health & Fitness
- **Secondary category:** Lifestyle
- **Content rating:** 4+ (no user-generated content, no external links to adult content)
- **Age rating questionnaire:** answer "no" to all — no third-party analytics, no ads, no in-app purchases (V1)

---

14. Known Risks & Mitigations

Risk	Likelihood	Severity	Mitigation
Foundation Models unavailable on user's device mid-session	Medium	High	Availability check before every session start. Graceful `AvailabilityGateView` for every failure state. Never crash.
SpeechAnalyzer accuracy degrades in noisy environments	Medium	Medium	Brief UI note: "Find a quiet space for best results." Silence threshold tunable.
3B model produces off-topic or low-quality responses	Medium	Medium	Short, directive prompts. @Generable constraints limit response shape. Test all phase prompts on device before shipping.
TTS Premium voices not downloaded on	Low	Medium	Implement fallback chain (Premium → Enhanced → Default). Check voice availability before session.

user's device			
Thermal throttling kills model mid-session	Low	High	Monitor thermal state via `ProcessInfo.thermalState`. If `.serious` or `.critical`, pause session gracefully with explanation.
App Store review flags the app as a medical device	Low	High	Wellness-only language throughout. Disclaimer prominent in onboarding. No clinical terminology in App Store listing. Health & Fitness category (not Medical).
SwiftData migration needed after model changes	Medium	Medium	Version SwiftData schemas from the start. Use `VersionedSchema` pattern.
User speaks during agent TTS (barge-in)	High	Low	On iOS this is a design choice, not a bug. Mic is muted during TTS by design. Phase 5 can add barge-in support if needed.
CoreHaptics unavailable (older device in future)	Low	Low	Wrap all haptic calls in availability check. Silent fallback if engine unavailable.

End of PRD v1.0

Coding agent instruction: Work top to bottom through build phases. Mark tasks 🔄 `IN PROGRESS` when started, ✅ `DONE` when complete and tested on a real device. Mark ❌ `BLOCKED` with a reason if stuck. Update the `Last Updated` field in document metadata after each session. Do not alter design specifications, prompt text, color tokens, or architecture decisions unless explicitly instructed by the product owner. When in doubt on Foundation Models API usage, refer to Section 5 — it takes precedence over general LLM assumptions.